



UNIVERSIDAD NACIONAL DE RÍO CUARTO

SISTEMAS OPERATIVOS

(Código 1965)

## Taller 5: Preemptive kernel tasks

Departamento de Computación

Facultad de Ciencias Exactas, Físico-Químicas y Naturales

**Alumno**  
Tomás Rodeghiero

**Fecha de entrega:** 4 de junio de 2026

## Índice

|                                        |           |
|----------------------------------------|-----------|
| <b>Objetivo</b>                        | <b>3</b>  |
| <b>Estructura de la entrega</b>        | <b>3</b>  |
| <b>Resolución por ejercicios</b>       | <b>3</b>  |
| <b>Resumen de cambios en el código</b> | <b>8</b>  |
| <b>Compilación y ejecución</b>         | <b>9</b>  |
| <b>Pruebas de ejecución</b>            | <b>9</b>  |
| <b>Conclusión</b>                      | <b>10</b> |

## Objetivo

Resolver el paso `05-kernel-tasks` del proyecto EDOS. El paso introduce *kernel threads* con desalojo por timer: cada CPU corre un `scheduler()` que selecciona una tarea `RUNNABLE` y hace un cambio de contexto hacia ella, y un `trap()` en `trap.c` desaloja la tarea cuando se le acaba el quantum. El `README.html` pide siete ejercicios: dos teóricos sobre overflow de timers (1 y 2), tres de análisis del código tal cual viene (3, 4 y 5) y dos de modificación: dar un quantum distinto a cada tarea (6) y recuperar la ejecución de un task tras una *invalid instruction exception* (7).

## Estructura de la entrega

- `05-kernel-tasks/`: paso original sin modificar, queda como referencia.
- `entrega/task.h`, `task.c`, `kmain.c`, `trap.c`: versiones modificadas de los cuatro archivos que modifiqué.
- `entrega/respuestas.txt`: respuestas a los siete ejercicios del `README.html`.
- `entrega/salida-kernel.txt`: salida observada con el kernel modificado.
- `entrega/salida-kernel-original.txt`: salida observada con el kernel sin modificar (sirve para comparar).
- `informe-taller5-kernel-tasks.pdf`: este informe.
- `README.md`: descripción breve.

## Resolución por ejercicios

### Ejercicio 1: overflow de un timer de 32 bits a 1 MHz

A 1 MHz (1 incremento por microsegundo) un registro de 32 bits desborda al cumplir  $2^{32}$  incrementos:

```
2^32 us = 4.294.967.296 us
        = 4.294.967,296 s
        ~= 71.582,79 min
        ~= 1.193,05 h
        ~= 49,71 dias
```

Esto es aproximadamente **49 días 17 horas 2 minutos 47 segundos**, el clásico *wrap* de los timers de 32 bits a 1 MHz.

## Ejercicio 2: ídem con 64 bits

Con 64 bits hay  $2^{64}$  valores. A la misma tasa:

```
2^64 us = 18.446.744.073.709.551.616 us
      ~= 1,8446 * 10^13 s
      ~= 5,8494 * 10^5 años
```

Es decir, más de **584.000 años** para que desborde. Por eso `CLINT_MTIME` en RISC-V (que es el contador subyacente del HAL de este paso) es de 64 bits: no hay que preocuparse del *wrap-around* durante la vida útil del sistema.

## Ejercicio 3: análisis de creación y manejo de tareas

(a) **Diagrama de transiciones.** Los estados están definidos en `task.h` (`UNUSED`, `RUNNABLE`, `RUNNING`, `WAITING`, `TERMINATED`). Las transiciones que aparecen en `task.c` son:

```

                                create_task()
UNUSED -----> RUNNABLE
    (slot libre + memset ctx +
     sp/ra iniciales + state=RUNNABLE)

                                scheduler()
RUNNABLE -----> RUNNING
    (selecciona slot RUNNABLE,
     carga quantum y context_switch)

                                yield()
RUNNING -----> RUNNABLE
    (vencimiento de quantum en trap();
     yield() pone RUNNABLE y llama a sched())

                                suspend(cond, lk)
RUNNING -----> WAITING
    (sleep(n) y otras condiciones;
     guarda wait_condition y llama a sched())

                                wakeup(cond) o wake_up_for_timer()
WAITING -----> RUNNABLE
    (la condicion se cumple o el sleep_ticks
     llega a 0)

                                terminate()
RUNNING -----> TERMINATED
    (imprime "Task X exit." y llama a sched();
     el slot queda ocupado pero ya no se elige)
```

`UNUSED` es el estado inicial de cada slot (`init_tasks()`). `TERMINATED` es absorbente: `scheduler()` sólo elige slots con `state == RUNNABLE`.

(b) **Política.** Round-robin con desalojo por timer. El scheduler recorre el arreglo `tasks[]` circularmente eligiendo el primer slot `RUNNABLE` y le entrega un quantum. Cuando vence, `trap()` llama a `yield()` y el scheduler reanuda el `for` desde el slot siguiente. No hay prioridades.

(c) **Idle.** Si no hay ninguna tarea `RUNNABLE`, el `while(true)` de `scheduler()` simplemente vuelve a recorrer `tasks[]` con las interrupciones habilitadas. Es un *busy idle loop*: la CPU sigue gastando ciclos hasta que llegue un timer interrupt o hasta que un `wakeup` (o el otro CPU) marque alguna tarea como `RUNNABLE`. No hay `wfi` ni una *idle task* dedicada.

## Ejercicio 4: dónde y cuándo se guarda el trap frame

El *trap frame* se guarda en la pila del task vigente, y el push lo hace `s_trap` en `arch.s` apenas se entra al handler:

`arch.s` (`s_trap`, parte de entrada)

```

1 s_trap:
2     addi    sp, sp, -30 * 4
3     sw      ra, 0 * 4 (sp)
4     sw      gp, 1 * 4 (sp)
5     sw      t0, 2 * 4 (sp)
6     ...
7     sw      s11, 28 * 4 (sp)
8     sw      sp, 29 * 4 (sp)
9     mv      a0, sp
10    call    trap

```

Es decir, ni bien la CPU salta a `s_trap` (por el vector configurado en `stvec`) se descuenta espacio del `sp` actual y se hace push de las 30 palabras que componen `struct trap_frame` (definida en `arch.h`). La dirección del frame queda en `sp` y se pasa como argumento (`a0`) a `trap()`. Cuando `trap()` retorna, `s_trap` restaura los registros desde la misma pila y libera el espacio con `addi sp,sp,+30*4` antes de `sret`.

Detalle importante: este kernel no usa pilas separadas por modo (no hay *swap* con `sscratch`), así que la pila usada es la del task vigente. Si durante `trap()` ocurre un `yield()` y se cambia de contexto a otra tarea, al volver a `s_trap` el `sp` apunta a otra pila (la del nuevo task), que tiene su propio trap frame; el restore sigue siendo coherente porque ese frame fue empujado por su propia entrada previa a `s_trap`.

## Ejercicio 5: compilar, ejecutar y analizar la salida

La compilación del original tira el warning esperado:

```
trap.c:8:30: warning: unused parameter 'tf' [-Wunused-parameter]
```

El `ld` también suma un warning de *LOAD segment with RWX permissions* que es propio del linker script del paso. Ambos se ignoran.

Salida del kernel original:

```

task A created!
task B created!
task C created!
ticks: 0, task A in cpu 1
ticks: 0, task B in cpu 0, going to sleep...
ticks: 0, task C in cpu 0
...
ticks: 3, task A in cpu 1
Task A in CPU 1 illegal instruction at address 80000790
Task A exit.
ticks: 3, task C in cpu 0
Task C in CPU 0 illegal instruction at address 80000790
Task C exit.
ticks: 4, task B in cpu 1, awake!
ticks: 4, task B in cpu 1, going to sleep...

```

Observaciones:

- A y C son tareas *compute-bound* (printf más un for ocupado) y se ejecutan en paralelo en las dos CPUs.
- B se duerme cuatro ticks (`sleep(4)`), por eso casi no aparece hasta que se cumple su tiempo.
- En `ticks==3`, A y C ejecutan `invalid_instruction()` (la pseudo-instrucción `unimp` de RISC-V): el handler imprime *illegal instruction* y termina la tarea con `terminate(-1)`. A partir de ahí sólo queda B ciclando *sleep/awake* cada cuatro ticks.

Probé con distintos `QUANTUM` (en `task.h`). Con `QUANTUM=1` cada tick desaloja, y A y C se alternan seguido (línea a línea entre `yields`). Con `QUANTUM=5` cada tarea imprime muchas líneas consecutivas antes de ceder, porque cada `printf` es mucho más corto que un tick (`T_INTERVAL = 5_000_000` ciclos en QEMU). No cambié `T_INTERVAL`; los interrupts llegan a un ritmo razonable, no hay nada que tunear.

## Ejercicio 6: quantum por tarea

Cambios en `task.h`, `task.c` y `kmain.c`.

(1) `task.h`. Agregué el campo `quantum` en `struct task` y cambié la firma de `create_task()` para que tome el `quantum` como tercer argumento:

`task.h` (cambios)

```

1 struct task {
2     ...
3     int          quantum;      // quantum (ticks) asignado a la tarea
4     int          ticks;       // ticks used in last cpu burst
5     ...
6 };
7
8 // create a RUNNABLE task with a per-task quantum (ticks)
9 struct task* create_task(char* name, task_function pc, int quantum);

```

(2) `task.c`. `create_task()` ahora asigna el quantum (con caída al `QUANTUM` del header si el caller pasa  $\leq 0$ ), y `scheduler()` carga ticks desde el quantum propio de la tarea en vez de la constante global:

`task.c` (cambios)

```
1 struct task* create_task(char* name, task_function f, int quantum) {
2     ...
3     task->quantum = (quantum > 0) ? quantum : QUANTUM;
4     ...
5 }
6
7 void scheduler(void) {
8     ...
9     next_task->ticks = next_task->quantum;    // antes: = QUANTUM;
10    ...
11 }
```

(3) `kmain.c`. Le doy a cada tarea un quantum distinto para notar la diferencia:

`kmain.c` (cambios en `kernel_main`)

```
1 create_task("A", f_a, 1);    // quantum bajo: cede rapido
2 create_task("B", f_b, 3);    // quantum mas grande
3 create_task("C", f_a, 2);    // intermedio
```

El resultado es que en `entrega/salida-kernel.txt` se ve que A aparece con muchos más task A yield CPU que C con task C yield CPU, justamente porque A consume su quantum (1) cada tick y C cada dos. B casi no se ve cediendo porque la mayor parte del tiempo está WAITING (durmiendo cuatro ticks).

Para chequear que sigue funcionando el caso por defecto, basta con llamar `create_task("X", f_x, 0)` (o con un valor negativo) y la tarea recibe el `QUANTUM` definido en `task.h`.

## Ejercicio 7: recuperar al task tras una invalid instruction

Cambio en `trap.c`. En la rama `ILLEGAL_INSTRUCTION`, en vez de llamar a `terminate(-1)` avanzo el `sepc` en 4 bytes y dejo seguir a la tarea:

`trap.c` (cambio en la rama de illegal instruction)

```
1 case ILLEGAL_INSTRUCTION:
2     if (task) {
3         printf("Task %s in CPU %d illegal instruction at address %x;"
4             " skipping it\n",
5             task->name, cpu_id, ra);
6         // Avanzo sepc 4 bytes para saltar la instruccion invalida.
7         // Como compilamos sin la extension C (-march=rv32imazicsr),
8         // todas las instrucciones miden 4 bytes.
9         // Al final, set_trap_ra(ra) escribe el nuevo sepc y el sret
10        // de s_trap salta a la siguiente instruccion del task.
11        ra += 4;
12    } else
13        panic("Kernel illegal instruction at %x on cpu %d\n", ra, cpu_id);
14    break;
```

**Por qué funciona.** La variable local `ra` se inicializa con `trap_ra()`, que lee `sepc` (el PC de la instrucción que disparó la excepción). Al final de `trap()` ya existe la línea `set_trap_ra(ra)`, que escribe `sepc` otra vez. Si entro a esa línea con `ra` apuntando a la siguiente instrucción, cuando `s_trap` haga `sret` la CPU saltará a la siguiente en vez de re-ejecutar la que falló.

**Por qué +4.** El README aclara: *remember that all RISC-V instructions are the same size*. El Makefile compila con `-march=rv32imazicsr` (sin la extensión C de instrucciones comprimidas de 2 bytes), así que todas las instrucciones miden 4 bytes. Esto se ve en `kernel.asm`: la pseudo `unimp` generada por `invalid_instruction()` aparece como una palabra de 4 bytes (`0xc0001073`) en `0x80000790`.

**Verificación.** En la salida del kernel modificado (`entrega/salida-kernel.txt`), tras los mensajes:

```
Task A in CPU 1 illegal instruction at address 80000790; skipping it
Task C in CPU 0 illegal instruction at address 80000790; skipping it
```

ninguna de las dos tareas termina: ya no aparece `Task A exit.` ni `Task C exit.`, y A y C siguen imprimiendo indefinidamente. La comparación contra `entrega/salida-kernel-original.txt`, donde las mismas dos tareas mueren y sólo queda B, es el contraste directo.

## Resumen de cambios en el código

Modifiqué cuatro archivos. Ninguno de los archivos de arquitectura (`arch.s`, `arch.h`, `arch.c`), ni el Makefile, ni `klib`, `console`, `spinlock` o `types.h` fueron modificados.

- `task.h`: nuevo campo `int quantum` en `struct task` y nuevo parámetro `int quantum` en `create_task()`.
- `task.c`: `create_task()` guarda el quantum recibido (con caída a `QUANTUM` si el valor es  $\leq 0$ ) y `scheduler()` usa `next_task->quantum` en vez de la constante `QUANTUM`.
- `kmain.c`: llamadas a `create_task` ahora pasan un quantum distinto para cada tarea (A=1, B=3, C=2).
- `trap.c`: en la rama `ILLEGAL_INSTRUCTION`, en lugar de `terminate(-1)` avanzo `ra += 4` para saltar la instrucción y dejo seguir a la tarea. También agregué `(void) tf;` para silenciar el warning de parámetro no usado, ya que el ejercicio 7 lo deja igual de no usado.



## Compilación y ejecución

Para probar la solución copiar los cuatro archivos modificados sobre el paso original, limpiar y compilar:

```
1 cp entrega/task.h entrega/task.c entrega/kmain.c entrega/trap.c \  
2   05-kernel-tasks/  
3 make -C 05-kernel-tasks clean PREFIX=riscv64-unknown-elf-  
4 make -C 05-kernel-tasks qemu PREFIX=riscv64-unknown-elf-
```

## Pruebas de ejecución

Comparé la salida del kernel original con la del kernel modificado para verificar ambos cambios.

**Kernel original:**

```
ticks: 3, task A in cpu 1  
Task A in CPU 1 illegal instruction at address 80000790  
Task A exit.  
ticks: 3, task C in cpu 0  
Task C in CPU 0 illegal instruction at address 80000790  
Task C exit.  
ticks: 4, task B in cpu 1, awake!  
...
```

**Kernel modificado:**

```
Task A in CPU 1 illegal instruction at address 80000790; skipping it  
ticks: 3, task A in cpu 1  
ticks: 3, task C in cpu 0  
Task C in CPU 0 illegal instruction at address 80000790; skipping it  
...  
task A yield CPU 1  
task C yield CPU 0  
ticks: 4, task B in cpu 0, awake!
```

Dos cosas se ven a simple vista:

1. Ya no hay `Task A exit.` ni `Task C exit.`: el handler saltó la instrucción inválida y la tarea siguió ejecutándose (ejercicio 7).
2. Los `task A yield CPU` aparecen con bastante más frecuencia que los `task C yield CPU`, lo cual es coherente con `A.quantum = 1` y `C.quantum = 2` (ejercicio 6).

## Conclusión

El paso es interesante porque junta varias piezas que ya estaban implementadas por separado en los talleres anteriores: *spinlocks* y arranque multi-CPU (taller 2), `printf` compartido (taller 3) y `context_switch` crudo (taller 4). Acá se ven funcionando juntos para sostener un planificador *round-robin* con desalojo por timer y un handler de traps que sobrevive a tareas mal portadas.

Los dos ejercicios de modificación ilustran de forma correcta dos conceptos: el *quantum* es un atributo **por tarea** y no una constante global (el campo nuevo en el PCB lo formaliza), y para que el `sret` no re-ejecute la instrucción inválida lo único que hace falta es modificar el `sepc` antes del retorno, no cambiar la pila ni el trap frame.